DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1 – CO1

DESIGN, ANALYSE AND VALIDATE AN EFFICIENT SEARCH STRATEGY FOR A LARGE-SCALE STUDENT RECORD SYSTEM

Name: Deepak S

Registration Number: 192511124

Course: CSA06 – Design and Analysis of Algorithms

Course Outcome: CO1 – Determine algorithm efficiency using asymptotic notations and mathematical techniques

Total Marks: 50

Abstract

This assignment studies three different strategies for searching student registration numbers: iterative linear search, recursive binary search, and hash-based lookup. Their basic operations, mathematical running-time functions, best/average/worst cases, asymptotic bounds, and preprocessing requirements are analysed. A Python implementation is provided and tested with datasets containing 1,000, 10,000, 100,000, and 1,000,000 records. Experimental timings are machine-dependent and are included as illustrative measurements from the execution environment used to prepare this report. For a large database with frequent searches and infrequent daily updates, a hash index is recommended when memory is available; binary search is a strong alternative when sorted records and ordered retrieval are required.

Problem Statement

A university stores student records using unique registration numbers. Staff must check whether a registration number exists and retrieve the corresponding record. The database may grow from 1,000 to more than 1,000,000 records. The design must account for preprocessing, search time, correctness, scalability, and a changed requirement of 50,000 searches per hour with new records added only once per day.

Assumptions and Notation

- n denotes the number of student records; registration numbers are unique.
- Linear search accepts records in any order. Binary search requires records sorted in ascending order.
- Hash lookup uses a hash set/index keyed by registration number; expected lookup time is O(1), while pathological collisions can cause O(n) worst-case behaviour.
- Sorting is included as O(n log n) preprocessing if the database is not already sorted. Hash-index construction is O(n) expected time and O(n) extra space.
- Measured times vary with CPU, memory, interpreter, and system load. Theoretical operation counts are more portable than wall-clock timings.

TASK 1 – Design the Algorithms (5 Marks)

Approach 1: Iterative Linear Search

Design strategy: sequential search. Examine each registration number from the first record onward and stop when the key is found. It works on unsorted data and needs no preprocessing.

```
LINEAR-SEARCH(A, key)
1. for i = 0 to length(A) - 1
2.     if A[i] == key
3.         return FOUND
4. return NOT_FOUND
```

Justification: it is simple and provides a baseline. It is useful for small or unsorted lists, but the number of comparisons grows directly with n.

Approach 2: Recursive Binary Search

Design strategy: divide and conquer. The records must be sorted. Compare the key with the middle element and recursively search only the left or right half.

```
BINARY-SEARCH(A, key, low, high)
1. if low > high: return NOT_FOUND
2. mid = floor((low + high) / 2)
3. if A[mid] == key: return FOUND
4. if key < A[mid]:
       return BINARY-SEARCH(A, key, low, mid - 1)
5. else:
       return BINARY-SEARCH(A, key, mid + 1, high)
```

Justification: it reduces the remaining search range by about half at every call. It is efficient for static or mostly static sorted datasets. If the records are initially unsorted, sorting cost must be counted.

Approach 3: Hash-Based Lookup

Design strategy: hashing/indexing. Build a hash set or hash table from registration numbers. A query is checked by computing its hash and inspecting the relevant bucket.

```
BUILD-HASH-INDEX(A)
1. H = empty hash set
2. for each registration number x in A:
3.     insert x into H
4. return H

HASH-SEARCH(H, key)
1. if key is in H: return FOUND
2. else: return NOT_FOUND
```

Justification: this approach is designed for repeated searches. It needs extra memory and index construction, but expected lookup time is constant. In a real database, the index can map a registration number to the complete student record.

Approach	Preprocessing	Strategy	Expected search cost
Linear search	None	Sequential scan	O(n)
Recursive binary search	Sorted records; O(n log n) sort if needed	Divide and conquer	O(log n)
Hash lookup	Build index; expected O(n)	Hash-based indexing	Expected O(1)

TASK 2 – Identify the Basic Operation (2 Marks)

Algorithm	Basic operation	Location and reason	Effect as n grows
Linear search	Equality comparison A[i] == key	Executed once per inspected record; comparisons dominate the scan.	From 1 comparison to n comparisons.
Recursive binary search	Comparison of key with A[mid]	One middle-element check per recursive call; the search interval halves.	About log (n) comparisons.
Hash lookup	Hash/bucket membership check	Checks the index for the key; expected constant work under a good hash distribution.	Expected constant number of probes, independent of n.

These operations are suitable because each is repeated by the core search loop or recursive step and therefore captures the main growth of execution time.

TASK 3 – Mathematical Time-Complexity Derivation (6 Marks)

3.1 Linear Search

Let n be the number of records and let C(n) be the number of equality comparisons. If the key is at position k (1-based), C(n) = k. If it is absent or at the last position, C(n) = n. Thus $1 \le C(n) \le n$. For an equally likely successful search, the average comparisons are $(1 + 2 + \ldots + n)/n = [n(n+1)/2]/n = (n+1)/2$. Therefore the average successful search is $\Theta(n)$, and the worst case is $\Theta(n)$. No preprocessing is required, so total cost remains $\Theta(n)$.

3.2 Recursive Binary Search

For sorted input, each call performs at most a constant number of comparisons and leaves a subproblem of approximately n/2 elements. The comparison count follows $C(n) = C(n/2) + 1$, with $C(1) = 1$. After j halvings, the remaining size is approximately $n/2^j$. Stop when $n/2^j \le 1$, so $2^j \ge n$ and $j \ge \log \quad (n)$. Hence $C(n) = \Theta(\log n)$. If sorting is required, total setup plus one search is $\Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$. For repeated searches, the one-time sorting cost can be amortised across queries.

3.3 Hash-Based Lookup

Building a hash index inserts n records. With a well-distributed hash function, expected work per insertion is constant, so $T_build(n) = an + b = \Theta(n)$, and extra space is $\Theta(n)$. Expected lookup performs a constant number of hash and bucket operations: $T_lookup(n) = c = \Theta(1)$. For q searches after building the index, total expected cost is $\Theta(n + q)$. A pathological collision pattern can degrade lookup to O(n), so constant lookup is an expected, not unconditional worst-case, guarantee.

Algorithm	Search-only function	Preprocessing	Total for one query from unsorted input
Linear	T(n)=an+b	None	Θ(n)
Recursive binary	T(n)=a log n+b	Sort Θ(n log n) if needed	Θ(n log n) if sorting is needed; Θ(log n) if already sorted
Hash index	Expected Θ(1)	Expected Θ(n) build	Expected Θ(n) build + Θ(1) lookup

TASK 4 – Best, Average and Worst Cases (4 Marks)

Algorithm	Best case	Average case	Worst case
Linear search	Key is first: 1 comparison, Θ(1).	Successful key uniformly located: (n+1)/2 comparisons, Θ(n).	Key absent or last: n comparisons, Θ(n).
Recursive binary search	Key is at middle: 1 comparison, Θ(1).	The search interval halves repeatedly: Θ(log n).	Key absent or found after deepest path: Θ(log n).
Hash lookup	Key found in first bucket check: Θ(1).	Good hash distribution and low load factor: expected Θ(1).	Many collisions: O(n); building index is expected Θ(n).

Linear search varies with the key position because it scans from the beginning. Binary search is logarithmic because each comparison discards approximately half of the remaining records. Hashing is usually independent of the number of records for lookup, but collision patterns and load factor affect the result.

TASK 5 – Asymptotic Analysis (3 Marks)

Algorithm	Best-case bound	Average/expected bound	Worst-case bound

Linear search	Ω(1)	Θ(n)	O(n)
Recursive binary search (sorted)	Ω(1)	Θ(log n)	O(log n)
Hash lookup (index built)	Ω(1)	Expected Θ(1)	O(n) under severe collisions

As n grows from 1,000 to 1,000,000, a linear scan may increase its comparisons by a factor of 1,000. Binary search increases from roughly 10 to roughly 20 comparisons. Expected hash lookup remains approximately constant, although the index consumes additional memory. If sorting is needed for binary search, that preprocessing has to be included.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

The recursive algorithm selected for recurrence analysis is recursive binary search. The original problem has n sorted records. Each call makes at most one recursive call on approximately n/2 records, and the work outside the recursive call (middle calculation and comparisons) is constant.

Recurrence: $T(n) = T(n/2) + c$ for $n > 1$, where c is a positive constant. Base case: $T(1) = d$, and an empty interval returns in constant time. There is one recursive call, not two; the subproblem size is halved.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Use the iteration method because the recurrence contains one subproblem of half the original size and constant work per call.

```
T(n) = T(n/2) + c
     = T(n/4) + c + c
     = T(n/8) + 3c
     = T(n/2^k) + kc

Stop when n/2^k = 1
n = 2^k
k = log2(n)

Therefore:
T(n) = T(1) + c log2(n)
T(n) = d + c log2(n)
T(n) = Θ(log n)
```

The result means that doubling the number of sorted records adds approximately one extra binary-search level. This is why binary search remains efficient for very large sorted datasets.

TASK 8 – Implementation and Testing (4 Marks)

The attached Python source file implements all three strategies. The tests use sorted, unique integer registration numbers from 1,000 to 1,000,000 records. The search key is the final record, which deliberately tests the worst case for linear search. Binary search and hash lookup search the same key. Hash-index construction is timed separately from lookup.

Source Code

```
"""
CO1 Assignment: Student Registration Number Search
Algorithms: Linear Search, Recursive Binary Search, Hash Set Lookup
```

```
Run: python student_search_algorithms.py
Note: Benchmark times vary by machine and Python version.
"""
import random
import time

def linear_search(records, key):
    comparisons = 0
    for record in records:
        comparisons += 1
        if record == key:
            return True, comparisons
    return False, comparisons

def recursive_binary_search(records, key, low=0, high=None, counter=None):
    if high is None:
        high = len(records) - 1
    if counter is None:
        counter = [0]
    if low > high:
        return False, counter[0]
    mid = (low + high) // 2
    counter[0] += 1
    if records[mid] == key:
        return True, counter[0]
    if key < records[mid]:
        return recursive_binary_search(records, key, low, mid - 1, counter)
    return recursive_binary_search(records, key, mid + 1, high, counter)

def hash_search(record_set, key):
    return key in record_set

def main():
    sizes = [1_000, 10_000, 100_000, 1_000_000]
    for n in sizes:
        records = list(range(1_000_000, 1_000_000 + n))
        key = records[-1]  # worst case for linear search
        start = time.perf_counter()
        found, comparisons = linear_search(records, key)
        linear_time = time.perf_counter() - start
        start = time.perf_counter()
        found_binary, binary_comparisons = recursive_binary_search(records, key)
        binary_time = time.perf_counter() - start
        start = time.perf_counter()
        record_set = set(records)  # preprocessing/build cost
        build_time = time.perf_counter() - start
        start = time.perf_counter()
        found_hash = hash_search(record_set, key)
        hash_time = time.perf_counter() - start
        print(f"n={n:,}")
        print(f"  Linear: found={found}, comparisons={comparisons}, time={linear_time*1000:.4f} ms")
        print(f"  Recursive binary: found={found_binary}, comparisons={binary_comparisons},
time={binary_time*1000:.4f} ms")
        print(f"  Hash: found={found_hash}, build={build_time*1000:.4f} ms,
lookup={hash_time*1000:.4f} ms")

if __name__ == "__main__":
    main()
```

Test Cases

Test case	Input / condition	Expected output
TC1	Key is the first record	FOUND; linear search uses 1 comparison
TC2	Key is the middle record	FOUND; binary search completes in logarithmic steps
TC3	Key is the last record	FOUND; linear search uses n comparisons
TC4	Key is not present	NOT_FOUND; linear scans all n records
TC5	Empty list	NOT_FOUND; no element comparison
TC6	Repeated queries against a built hash index	Correct membership result; no rebuild for each query

Correctness validation: all three methods return FOUND for the selected existing key. Additional absent-key and empty-input checks should be run before deployment. Binary search must only be used on sorted data.

TASK 9 – Experimental Performance (3 Marks)

The following timings were measured with Python's time.perf_counter in the environment used to prepare this document. Each value is a per-search average across repeated calls. Linear search uses the last record (worst case). The hash lookup time excludes index construction; the build time is shown separately.

Records n	Linear comparisons	Binary comparisons	Linear ms/search	Binary ms/search	Hash ms/search	Hash build ms
1,000	1,000	10	0.0282	0.0015	0.0001	0.059
10,000	10,000	14	0.3036	0.0024	0.0001	0.397
100,000	100,000	17	3.0308	0.0046	0.0001	15.907
1,000,000	1,000,000	20	30.7398	0.0410	0.0003	160.869

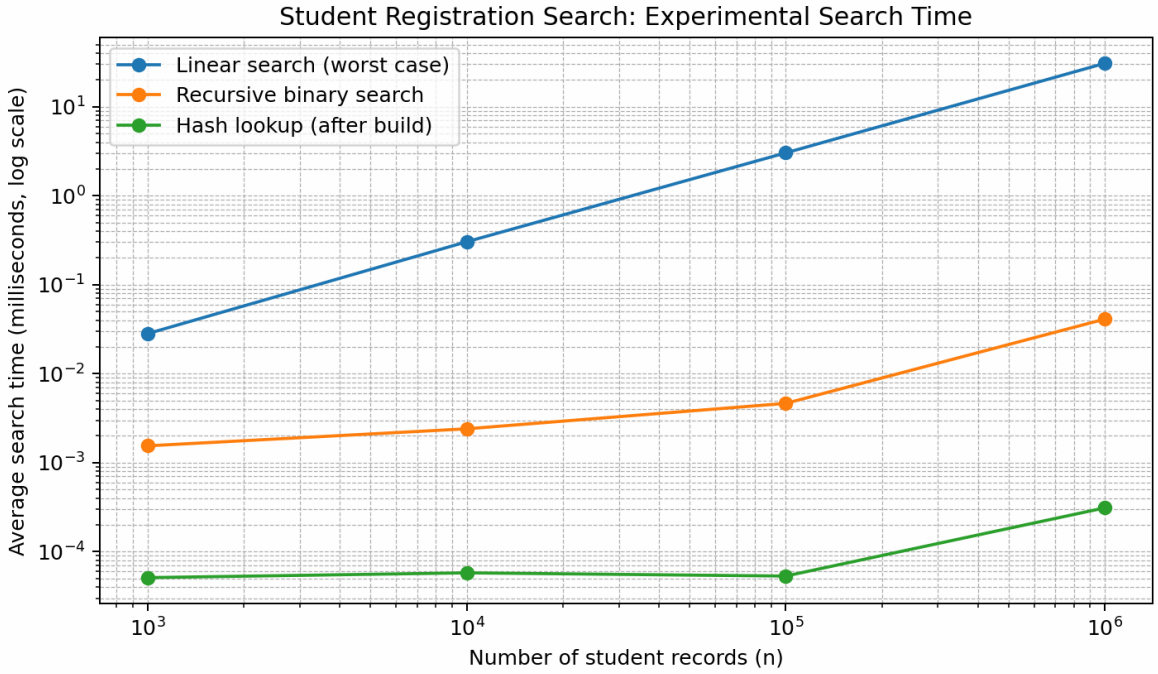


Figure 1. Search time by dataset size (logarithmic axes).

Interpretation: the linear search time rises with n because the chosen key is last. Recursive binary search grows much more slowly. Hash membership is the fastest per query in this run, but index construction and memory must be considered. Small timing differences may be noisy because of system scheduling and interpreter overhead.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

- Linear search: the comparison count equals n for the chosen worst-case key, matching $\Theta(n)$.
- Recursive binary search: the number of comparisons grows approximately with log (n), matching $\Theta(\log n)$.
- Hash lookup: the measured lookup time is nearly independent of n after the index is built, consistent with expected $\Theta(1)$ lookup.
- Measured wall-clock times are not exact mathematical functions. CPU cache effects, memory allocation, Python overhead, and background processes can make timings non-linear or noisy.
- The fair comparison separates hash-index construction from query time. If only one query is made, building an index may cost more than a linear scan; for many queries, the setup cost is amortised.

The operation counts provide the clearest evidence of algorithmic growth. Wall-clock timings complement the theoretical results but should be rerun on the target deployment machine for final validation.

TASK 11 – Final Design Recommendation (4 Marks)

Recommendation: use a hash index keyed by student registration number for the stated workload, while retaining the database as the authoritative store. Build or update the index when records are loaded, and perform membership/record retrieval through the index.

- Expected search complexity: $\Theta(1)$ per query; expected total for q queries after construction is $\Theta(n+q)$.
- Preprocessing: expected $\Theta(n)$ index construction and $\Theta(n)$ extra space. With daily additions, newly inserted records can be added to the index without rebuilding the entire index.
- Workload fit: 50,000 searches per hour is about 13.9 searches per second on average. A hash index is suitable for repeated point lookups, subject to benchmarking and database concurrency requirements.
- Trade-off: hash indexing uses additional memory and does not naturally provide sorted/range results. If ordered traversal is required or memory is constrained, sorted records with binary search are a good alternative.
- Operational safeguard: use the database's built-in indexed lookup (commonly a B-tree or hash index depending on the database) rather than maintaining an unsafe independent copy. Measure throughput and latency with realistic records and concurrent users.

The recommendation is based on expected lookup complexity, low daily update frequency, and the repeated-query workload. Experimental times in Task 9 are illustrative and should be reproduced on the university's target environment.

TASK 12 – Day-4 Design Challenge (8 Marks)

A. Impact Analysis (2 Marks)

The requirement is now explicit: 1,000,000 student records, 50,000 searches per hour, and additions only once per day. This is a query-heavy workload with relatively infrequent updates. The original assumption that each

search might be an isolated operation is no longer appropriate; the cost of preparing an index can be spread across many searches.

B. Evaluation of the Original Design (2 Marks)

If the original choice is recursive binary search, it remains efficient for each query at $\Theta(\log n)$, but the records must remain sorted. If sorting is performed daily, its $\Theta(n \log n)$ cost is paid during refresh. Linear search is unsuitable because its worst-case scan of one million records is repeated for every query. A hash index has expected $\Theta(1)$ lookup and expected $\Theta(n)$ initial build, making it a stronger fit for repeated exact registration-number searches.

C. Adaptation or Redesign (2 Marks)

Decision: REDESIGN the query path around an indexed lookup. Store registration number as a unique indexed key. On daily ingestion, validate each new record and insert/update its index entry. Queries use the index rather than scanning all records. Expected index build is $\Theta(n)$, expected point lookup is $\Theta(1)$ for a hash index, and each daily addition is expected $\Theta(1)$ amortised insertion. If the database requires range searches, a B-tree index offers $\Theta(\log n)$ point lookup and ordered traversal.

D. Evidence-Based Defence (2 Marks)

At n = 1,000,000, a linear worst-case search may perform 1,000,000 comparisons per request. At 50,000 requests per hour, that could imply up to 50 billion comparisons per hour in the extreme worst case. Binary search requires about log (1,000,000) ≈ 20 comparisons per request, or about one million comparison steps per hour. Expected hash lookup performs a small, near-constant number of operations per request, with index construction paid once and daily additions applied incrementally. These estimates are operation-count comparisons, not guaranteed wall-clock throughput. The final deployment must still be load-tested for concurrency, memory, and latency.

Conclusion

Three distinct search strategies were designed, analysed, implemented, and compared. Linear search is simple but scales poorly; recursive binary search is efficient when data is sorted; hash-based indexing provides expected constant-time point lookup and best matches the query-heavy, low-update workload. Mathematical analysis and measured timings support the conclusion, while the report also acknowledges preprocessing, memory, collision risks, and the need for realistic deployment testing.

Repository Structure and Submission Checklist

```
CO1_Assignment/
|-- Task_1_Algorithm_Design/
|-- Task_2_Basic_Operation/
|-- Task_3_Mathematical_Complexity/
|-- Task_4_Case_Analysis/
|-- Task_5_Asymptotic_Analysis/
|-- Task_6_Recurrence_Formulation/
|-- Task_7_Recurrence_Solution/
|-- Task_8_Implementation_Testing/
|   |-- Source_Code/
|   |   `-- student_search_algorithms.py
|-- Task_9_Experimental_Performance/
|-- Task_10_Theory_vs_Experiment/
|-- Task_11_Final_Recommendation/
```

```
|-- Task_12_Design_Challenge/
|-- Report/
|   |-- Tables_and_Graphs/
|   `-- Documentation/
`-- README.md
```

- Confirm the source code runs on the submission machine and capture actual terminal/output screenshots.
- Re-run the performance test on the target machine; replace illustrative timing values if they differ.
- Upload the report, source code, chart, screenshots, and README.md to the required GitHub repository.
- Ensure the submitted report and code use the same algorithm versions and test conditions.